Business Requirements Document (BRD)

Project Name: SecureCare Clinic Management System
(CMS) **Version:** 1.0 **Date:** January 25, 2026 **Document Status: FINAL**

1. Executive Summary

The **SecureCare CMS** is an application designed to manage the core operations of a medical clinic. Beyond its functional utility, this system serves as an **Architectural Reference Implementation**, explicitly designed to demonstrate advanced software engineering concepts including **Onion Architecture**, **CQRS (Command Query Responsibility Segregation)**, **Domain-Driven Design (DDD)**, and granular **Role-Based Access Control (RBAC)**.

The primary success metric is the system's ability to enforce strict security boundaries between different user personas while maintaining a seamless user experience.

2. Project Scope

2.1 In-Scope (Phase 1)

1. Staff & Security Management

- **Role-Based Access for Staff:** The system must strictly control what information staff members (Doctors, Receptionists, Admins) can access based on their assigned roles.
- **Secure Access:** The system must provide a secure login mechanism for all users to prevent unauthorized access to sensitive medical data.

2. Patient Registry

- **Patient Record Management:** The ability to register new patients and maintain up-to-date demographic information (Name, Contact, Emergency details).
- **Search & Retrieval:** Staff must be able to search for existing patients within the system to avoid duplicate registrations.

3. Clinic Scheduling & Appointments

- **Appointment Booking:** The capability to schedule patient visits with specific doctors.
- **Availability Management:** The system must detect and prevent scheduling conflicts (double-booking) automatically.
- **Appointment Lifecycle:** Ability to track an appointment from initial request to confirmation and final completion.

4. Medical Consultation Records

- **Prescription Recording:** Doctors must be able to digitally record prescriptions and consultation notes for completed appointments.
- **Record Integrity:** The system must ensure that once medical advice is finalized, it is preserved as an unchangeable permanent record.

5. Compliance & Auditing

- **Activity Tracking:** The system must maintain a history log of important actions (such as deleting a user or changing a role) for accountability and security compliance.

2.2 Out-of-Scope (for this phase)

- **Financial Processing:** The system will not handle billing, invoicing, or insurance claims processing.
- **Inventory Management:** The system will not track pharmacy stock levels or medication inventory.
- **Patient Portal:** There is no requirement for a public-facing interface for patients to self-register or book appointments (Staff-facing only for now).

3. User Roles & Permissions Matrix

The system is defined by its ability to serve distinct personas.

Role	Access Level	Primary Goal	Critical Restriction
System Admin	**Tier 0 (Root)**	Maintain system health and user accounts.	**CANNOT** view patient medical history or prescriptions (Privacy Compliance).
Doctor	**Tier 1 (Clinical)**	Treat patients and manage medical records.	Can **ONLY** view records for patients they have treated or are scheduled to treat.
Receptionist	**Tier 2 (Ops)**	Facilitate patient flow and scheduling.	**CANNOT** view clinical notes or prescriptions.
Patient	**Tier 3 (Public)**	View personal history and book slots.	**STRICT Isolation**: Cannot see any data belonging to other patients.

4. Functional Requirements

4.1 Security Module (Authentication & Authorization)

- **FR-SEC-01:** The system **must** use stateless authentication via JSON Web Tokens (JWT).
- **FR-SEC-03:** Passwords **must** be hashed using recognized cryptographic algorithms (e.g., BCrypt, Argon2).
- **FR-SEC-04:** Authorization **must** be implemented via Policies (e.g., Policy="CanPrescribeMedication") rather than hard roles to allow for future flexibility.

4.2 Patient Management Module

- **FR-PAT-01:** Receptionists and Admins **must** be able to register new patients.
- **FR-PAT-02:** The system **must** validate uniqueness of the National ID/SSN and Email address.
- **FR-PAT-03:** Patients **must** be able to update their own contact details (Phone/Address) but not their medical identifiers.

4.3 Appointment Scheduling Module

- **FR-APT-01:** The system **must** provide a way to query "Available Slots" for a specific Doctor on a specific Date.
- **FR-APT-02:** The system **must** enforce concurrency control to prevent two users from booking the same slot simultaneously.
- **FR-APT-03:** An appointment **must** flow through legally defined states: Pending → Confirmed → Completed (or Cancelled).

4.4 Clinical/EMR Module

- **FR-CLI-01:** Doctors **must** be able to generate a Prescription record linked to a Completed appointment.
- **FR-CLI-02:** Once a Prescription is finalized, it **must** be immutable (Read-Only). Corrections require a new entry.

5. Non-Functional Requirements (Architecture / Technical)

5.1 Architectural Standards

- **NFR-ARC-01:** The solution **must** strictly follow **Clean Architecture (Onion)**:
    - *Core*: Entities & Interfaces (No Dependencies).
    - *Application*: Use Cases & Business Logic.

    - *Infrastructure*: DB, File System, External APIs.
    - *API*: Controllers & Entry Points.
- **NFR-ARC-02:** Read and Write operations **should** be separated using the **CQRS Pattern** (via MediatR library).
- **NFR-ARC-03:** Domain Logic **must** be unit testable without database dependencies.

5.2 Performance & Reliability

- **NFR-PER-01:** API response time for "Get Available Slots" must be under 300ms.
- **NFR-REL-01:** All "Write" operations (Create/Update/Delete) must be wrapped in Database Transactions to ensure data integrity.

6. Assumptions

1. The application will use a Relational Database (SQL Server or PostgreSQL).
2. Email notifications (for appointment confirmations) will be logged/mocked in the initial development phase.

7. Acceptance Criteria (Definition of Done)

The project is considered complete only when the following End-to-End scenarios pass:

7.1 Security & Admin Validation

- **Admin Role Assignment**: An Admin can successfully create a new user and assign them the Doctor role.
- **Access Denial**: A Doctor receives a **403 Forbidden** error when attempting to access the POST /api/users (Create User) endpoint.

7.2 Receptionist Workflow Validation

- **Proxy Booking**: A Receptionist can successfully book an appointment *on behalf of* a Patient.
- **Privacy Check**: A Receptionist browsing a Patient's profile **cannot** see the "Prescriptions" tab or API data.

7.3 Doctor Workflow Validation

- **My Schedule**: A Doctor logging in sees only their own upcoming appointments, not those of colleagues.

- **Clinical Documentation**: A Doctor can successfully append a Prescription to a specific Appointment ID.

7.4 Patient Workflow Validation

- **Self-Service**: A Patient can log in to view their own appointment history.
- **Data Isolation**: A Patient attempting to access /api/appointments/{id} for an appointment belonging to another user receives a **403/404** error.